University Mohammed VI Polytechnic

College of Computing

Lab 2: Relational Schema of the RLMS

Functional Dependencies and Team Report

Data Management Course

UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering **Session:** Fall 2026

Team Information

Team Name	.....
Member 1	Malak Nhami
Member 2	Marwa Mazouz
Member 3	Mohamed Moussaid
Member 4	Sara Lamnaiy
Repository Link	`https://github.com/malak-3108/project`

1. Functional Dependencies of the RLMS Schema

The functional dependencies (FDs) below are derived from the relational schema of Lab 2. For every relation we list the FD given by its primary key, and then the additional FDs coming from candidate keys or from business rules.

Notation and convention. Attribute sets are written without braces. Many other FDs follow from Armstrong's axioms (reflexivity, augmentation, transitivity), for example the dependency from a primary key to every combination of the attributes it already determines. We *do not* list those implied FDs, since their number is exponential in the number of attributes. We list only the FDs that are generated by the primary key and by the extra keys and rules, which is enough to recover all the others through the axioms.

Relation	Functional dependencies
Person	$pid \to firstName, lastName, email, affiliation, startDate, endDate$ $email \to pid$ (*email is a candidate key*; hence email also determines every attribute of Person)
Student	$pid \to program$
Employee	$pid \to phone, office, supervisorPid$
Academic	only $pid \to pid$ (trivial; the relation has a single attribute)
NonAcademic	only $pid \to pid$ (trivial)
Faculty	$pid \to position$
Administrative	$pid \to position$
Technical	$pid \to position$
Advises	no non-trivial FD (all-key relation on $(academicPid, studentPid)$)
Laboratory	$labId \to name, building, roomNumber, discipline, supervisorPid$ $building, roomNumber \to labId$ (*(building, roomNumber) is a candidate key*)
Attached	no non-trivial FD (all-key relation on $(pid, labId)$)
ResearchProject	$code \to title, startDate, endDate, status$
Participates	$pid, projectCode \to role$
Budget	$budgetLine \to amountGranted, amountDisbursed, startDate, endDate, managerPid$
FundsPrj	no non-trivial FD (all-key relation)
FundsLab	no non-trivial FD (all-key relation)
EquipmentModel	$modelId \to$ $commercialName, manufacturer, category, requiredEnvironment, trainingMandatory$
EquipmentUnit	$serialNo \to acquisitionDate, purchaseCost, status, portable, modelId, labId$
Certification	$code \to title, issuingAuthority, validityPeriod, safetyLevel$
Requires	no non-trivial FD (all-key relation on $(modelId, certCode)$)
Holds	$pid, certCode \to issueDate, expirationDate, grade$
Reservation	$resId \to$ $submissionTS, plannedStart, plannedEnd, purpose, status, madeByPid, projectCode, approverPid$ $submissionTS, madeByPid \to resId$ (*candidate key*: a person cannot make two reservations at the exact same time)
Reserves	no non-trivial FD (all-key relation on $(resId, serialNo)$)
Maintenance	$serialNo, startTS \to endTS, type, description, cost, outcome, technicalPid$ $description \to type$ (business rule: the details in the description directly lead to the type of the problem that requires maintenance)
CalibrationRecord	$serialNo, calibDate \to calibrationType, result, nextDueDate, remarks$
Consumable	$consId \to name, unitOfMeasure, hazardLevel, reorderThreshold$
Supplier	$suppId \to name, contactEmail, phone$
Supplies	$suppId, consId, labId \to unitPrice$
Stocks	$labId, consId \to$ $quantityOnHand, lastRestockDate, storageCondition, technicalPid, monitoringSince$
Consumes	$resId, labId, consId \to quantityUsed$

Remark. Because of the extra rule $description \to type$, the relation **Maintenance** is not in BCNF (nor 3NF), since *description* is not a superkey and *type* is not a prime attribute. We kept this FD because it reflects our reading of the business rule; a possible decomposition would move $(description, type)$ to its own relation.

2. Team Report

2.1 Role of each team member

The whole project was carried out as a group, without a fixed split of the work. Every member read the ER diagram of Lab 1 and the Lab 2 specification, and proposed their own interpretation of the entities, keys and constraints. We then shared our proposals, explained the reasoning behind each one, discussed the differences, and finally agreed on a *single common answer* for every point. All members therefore took part equally in the conversion of the ER diagram, in the identification of candidate keys and functional dependencies, and in the writing and proofreading of this report.

2.2 Key challenges and how they were resolved

- **Deciding whether an attribute (or set of attributes) is a candidate key.** For several tables the business rules are not explicitly defined, so it was unclear whether some attributes were unique. Each member argued from a realistic situation, and we kept a candidate key only when a real-world rule justified it. Examples: *email* identifies a Person; *(building, roomNumber)* identifies a Laboratory; *(submissionTS, madeByPid)* identifies a Reservation.
- **Ambiguous meaning of some attributes.** In this context we did not know the exact meaning of several attributes. For example, *commercialName* could be the name of a product or of its type, and the attribute *name* appears in many entities (Laboratory, Consumable, Supplier) and could designate the item itself or its category. We resolved it by reading each attribute within its own entity: *commercialName* is the market name of the equipment model, and *name* is the name of the entity that owns it. We did not assume that these names are unique, which is why they never appear on the left-hand side of an FD.
- **Number of FDs.** Listing every FD implied by Armstrong's axioms (for instance a primary key determining every combination of the other attributes) would be exponentially large. We decided to list only the FDs generated by the primary keys, candidate keys and business rules.
- **Maintenance type.** We had to decide whether *type* depends only on the key or also on the description; we chose the latter (see assumptions).

2.3 Justification of the conversion choices

- **ISA hierarchies:** one relation per superclass and per subclass, where the subclass key is also a foreign key to its parent. This avoids NULLs from specialised attributes and keeps the hierarchy explicit.
- **ReportsTo:** mapped with a nullable *supervisorPid* in Employee, since an employee has at most one supervisor; NULL is needed for the Dean.
- **Many-to-one relationships** (Supervises, Manages, InstanceOf, LocatedIn, MadeBy, For, Approves, DoneBy) are absorbed into the "many" side to avoid extra tables. Approves is nullable because some reservations need no approval.
- **Many-to-many relationships** (Advises, Attached, Participates, FundsPrj, FundsLab, Requires, Holds, Reserves) get their own table whose primary key is the combination of the keys of the participants; relationship attributes (role, issueDate, ...) are stored there.
- **Weak entities** (Maintenance, CalibrationRecord) take the owner key plus the partial key (*startTS*, *calibDate*) as primary key; the identifying relationships need no table.
- **Supplies** is a separate ternary table, because a supply arrangement can exist even when a laboratory does not currently stock the consumable.
- **Monitors** (aggregation over Stocks) is stored in Stocks as *technicalPid* and *monitoringSince*, because each stock occurrence has exactly one monitor.
- **Consumes** references Stocks($labId, consId$), so we know from which laboratory the quantity was taken.

2.4 Assumptions

- *email* is unique, so it is a candidate key of Person.
- Two laboratories cannot share the same ($building, roomNumber$); this pair is a candidate key of Laboratory.
- A person cannot make two reservations at exactly the same time, so ($submissionTS, madeByPid$) identifies a Reservation.
- In Maintenance, the description determines the type of problem ($description \rightarrow type$).
- The attributes *name* and *commercialName* are not assumed unique.
- A Person has at most one role in a given project, and an EquipmentUnit appears at most once in a reservation.

- The constraints that the schema cannot enforce (ISA disjointness and coverage, Dean rules, certification checks, etc.) are handled by triggers, CHECK constraints or application logic, as listed at the end of the Lab 2 solution.